

PROMETEO

Acceso a BBDD con JDBC

Índice

01

Introducción

JDBC

Componentes y funcionamiento

02

Implementación

Estructura básica

Buenas prácticas



1

Introducción

PROMETEO

01

JDBC

JDBC (Java Data Base Connectivity) es una API estándar de Java que permite a las aplicaciones **conectarse, consultar y actualizar** datos en **bases de datos relacionales**.

Actúa como un puente, permitiendo el uso de sentencias SQL desde Java para gestionar información, **independiente del sistema operativo o gestor** de base de datos específico.

El acceso a bases de datos es fundamental en la mayoría de las aplicaciones empresariales y web. JDBC ofrece un mecanismo estandarizado y flexible para interactuar con distintos sistemas de bases de datos, favoreciendo la portabilidad y la escalabilidad de las aplicaciones.

01

JDBC

Un **ORM** (Object-Relational Mapping) en Java es una técnica y herramienta que actúa como puente entre el paradigma orientado a objetos y el relacional.

- **Mapea automáticamente clases Java a tablas** de base de datos y **atributos a columnas**.
- Permite la manipulación de los datos utilizando objetos Java en lugar de consultas SQL más complejas.

Aunque actualmente se utilizan ORMs, como Hibernate o JPA, **JDBC** sigue siendo la base tecnológica subyacente y es **útil para consultas complejas** ya que JDBC proporciona acceso a la base de datos de bajo nivel, permitiendo la **optimización de consultas y el control total sobre las conexiones y las transacciones**, lo que lo hace adecuado para **aplicaciones de alto rendimiento**.

Componentes y funcionamiento

El proceso básico implica **establecer** la conexión mediante una URL, **ejecutar** la sentencia SQL y **gestionar** los resultados.

➤ **Driver JDBC:**

Cada base de datos requiere un driver específico que implementa la API JDBC. Este driver actúa como puente entre la aplicación Java y la base de datos (MySQL, Oracle, PostgreSQL, etc.)

➤ **URL de Conexión:**

Especifica la dirección del servidor de la base de datos, puerto y la base de datos a la que se desea conectar. Su formato varía según el driver y el SGBD, por ejemplo:

- MySQL -> jdbc:mysql://localhost:3306/nombre_bbdd
- Oracle -> jdbc:oracle:thin:@//localhost:1521/servidor.nombre

01

Componentes y funcionamiento

Las interfaces **Connection**, **Statement**, **PreparedStatement** y **ResultSet** se encuentran en el paquete **java.sql** y son clave en el uso de JDBC.

Se implementan en cada Driver de la base de datos, lo que permite trabajar con independencia del motor de la base de datos.

- **Connection:** Interfaz para establecer una sesión de conexión con la base de datos. Representa la conexión activa a la base de datos.
- **Statement:** Interfaz utilizada para ejecutar sentencias SQL estáticas

01

Componentes y funcionamiento

- **PreparedStatement:** Interfaz que extiende Statement para ejecutar sentencias SQL parametrizadas, mejorando la seguridad (contra inyección SQL) y el rendimiento.
- **ResultSet:** Interfaz que representa la tabla de datos generada tras la ejecución de una SELECT. Almacena los resultados devueltos por una consulta SELECT, permitiendo recorrer y procesar los datos.

A group of people are working in a modern office setting. In the foreground, a man with grey hair and a beard is smiling and looking back over his shoulder. He is sitting at a desk with a laptop. Behind him, several other people are also working and smiling. The office has a warm, modern feel with wooden desks and plants.

2

Implementación

PROMETEO

02

Estructura básica

1. **Establecer la Conexión:** Se utiliza *DriverManager.getConnection()* pasando la URL, usuario y contraseña.
2. **Crear y Ejecutar Sentencias:** Se crea un objeto *Statement* o *PreparedStatement* para ejecutar consultas o actualizaciones.
3. **Procesar Resultados:** Con un *ResultSet* se recorre el resultado de una consulta y se extraen los datos.
4. **Cerrar Recursos:** Es fundamental cerrar los objetos *ResultSet*, *Statement* y *Connection* para liberar recursos, preferiblemente utilizando la estructura *try-with-resources*.

02

Buenas prácticas

- **Manejo de Excepciones:** Capturar y gestionar las excepciones SQLException para identificar y resolver problemas de conexión y ejecución.
- **Uso de PreparedStatement:** Para evitar inyecciones SQL y mejorar el rendimiento es recomendable usar PreparedStatement en lugar de concatenar directamente los valores en las sentencias SQL.
- **Cierre de Recursos:** Siempre cerrar los objetos Connection, Statement y ResultSet para evitar fugas de memoria y bloqueo de recursos en la base de datos.
- **Pool de Conexiones:** En aplicaciones de gran escala se recomienda utilizar un pool de conexiones para mejorar el rendimiento y la gestión de recursos.



PROMETEO